Assignment 2: Training Language Models on Your Topic of Interest

Overview

In this assignment, you will select a topic of interest, gather relevant data, train a small language model from scratch, and apply LoRA (Low-Rank Adaptation) fine-tuning to a GPT-2-like model. All model training and evaluation will be performed on the LUNARC. You will have LLM API access provided, which you may use *only* to help generate or augment your dataset. You will evaluate the model's performance, compare results before and after fine-tuning, and present your findings. This hands-on project will provide you with practical experience in working with language models and modern fine-tuning techniques end-to-end.

What you will NOT be graded down for

- The from-scratch model produces ungrammatical text.
- The from-scratch model having higher perplexity than base GPT-2 (this is expected)
- Generations from either model occasionally repeating or drifting off-topic.

What you WILL be graded down for

- An implementation that does not reproduce end-to-end offline from the provided scripts.
- Training loss curves that are flat or increasing.
- No measurable improvement from LoRA fine-tuning over base GPT-2-like models on your eval set.
- A report that overstates what the tiny model can do, or that reports numbers you cannot reproduce from your checkpoints.

Learning Objectives

- Define a specific topic domain suitable for language model training and fine-tuning
- Gain experience with dataset construction, including using a provided LLM API for data generation/augmentation
- Implement and train a small language model from scratch
- Apply LoRA fine-tuning to a pre-trained GPT-2-like model
- Evaluate model performance using appropriate metrics, computed locally
- Communicate technical findings effectively through written reports and presentations

Assignment Steps

Step 1: Topic Selection

Choose a specific topic or domain of interest for training a language model and fine-tuning your mini GPT-2 (or similar) model. Your topic should be:

- Specific enough to yield coherent results
- Broad enough to provide sufficient training data

Examples of potential topics:

- Marketing writing
- Recipe instructions
- Poetry from a particular era or author
- Legal documents
- Technical documentation for a software framework

Step 2: Data Collection (1 week)

Construct a dataset appropriate for your selected topic. You may use the **provided LLM API** to generate or augment samples — this is the *only* external API used in the assignment. All subsequent steps (training, evaluation, inference) must run locally and offline.

Purpose of this process: Assume that a generic small language model or a GPT-2-like model is not an expert in your topic of interest. After training (both a from-scratch language model and a LoRA fine-tuning process), your final models should perform noticeably better on this topic.

Requirements:

- Collect at least 500 text samples related to your topic. You may start from openly available documents on your chosen topic and/or use the provided LLM API to generate/augment the dataset. Be mindful about the quality of the data, otherwise it's ***garbage in, garbage out***.
- Ensure proper formatting and preprocessing of your dataset following this example: https://drive.google.com/file/d/1yjV36wU0PuIii9soRmhhi9qSH3rXZRAM/view?usp=sharing
- Split your data into training (80%) and evaluation (20%) sets
- Save the final dataset locally (e.g., as JSON/JSONL or plain text) so that all subsequent steps require no network access
- Document your data collection process. Max 1-page in the final report, including:
    - Sources used (documents, LLM API prompts, etc.)
    - Prompts / search terms
    - Data cleaning steps
    - Final dataset statistics (number of samples, average length, vocabulary size, etc.)

Step 3: Model Training and LoRA Fine-Tuning

You will train **two models**:

1. **A small language model from scratch.** Implement a custom (mini) language model in PyTorch (or a framework of your choice) and train it on your collected dataset.
2. **A LoRA fine-tune of a pre-trained GPT-2-like model.** Download the pre-trained weights once (e.g., via the Hugging Face model cache) and then perform LoRA fine-tuning.

Requirements:

- Implement a custom small LM (architecture, tokenizer choice, training loop)
- Apply LoRA fine-tuning to a pre-trained GPT-2-like model, configuring rank, alpha, target modules, and dropout
- Configure and justify appropriate hyperparameters (learning rate, batch size, sequence length, etc.) for both models
- Train each model for a minimum of 3 epochs, or more depending on quality
- Monitor and record training metrics (loss, perplexity, etc.), log to a local file or tool such as TensorBoard
- Save checkpoints of your models at different training stages

Step 4: Model Evaluation

Evaluate both of your trained models **locally**, using your held-out evaluation set.

Requirements:

- Compute standard metrics (e.g., perplexity, loss) on your evaluation set using your own scripts
- Generate sample outputs from each model and assess qualitatively
- Compare:
    - Your from-scratch small LM against the base pre-trained GPT-2-like model
    - Your LoRA fine-tuned model against the base pre-trained GPT-2-like model
- Analyze strengths and weaknesses of each model in capturing domain-specific content

Step 5: Report and Presentation

Written Report (4 pages maximum):

Your report should include:

- Introduction to your chosen topic and its relevance
- Description of your data collection and preprocessing methodology (including any use of the provided LLM API for data generation)
- Custom small LM implementation and configuration
- LoRA fine-tuning approach and configuration
- Quantitative and qualitative evaluation results, comparing both trained models against the base model

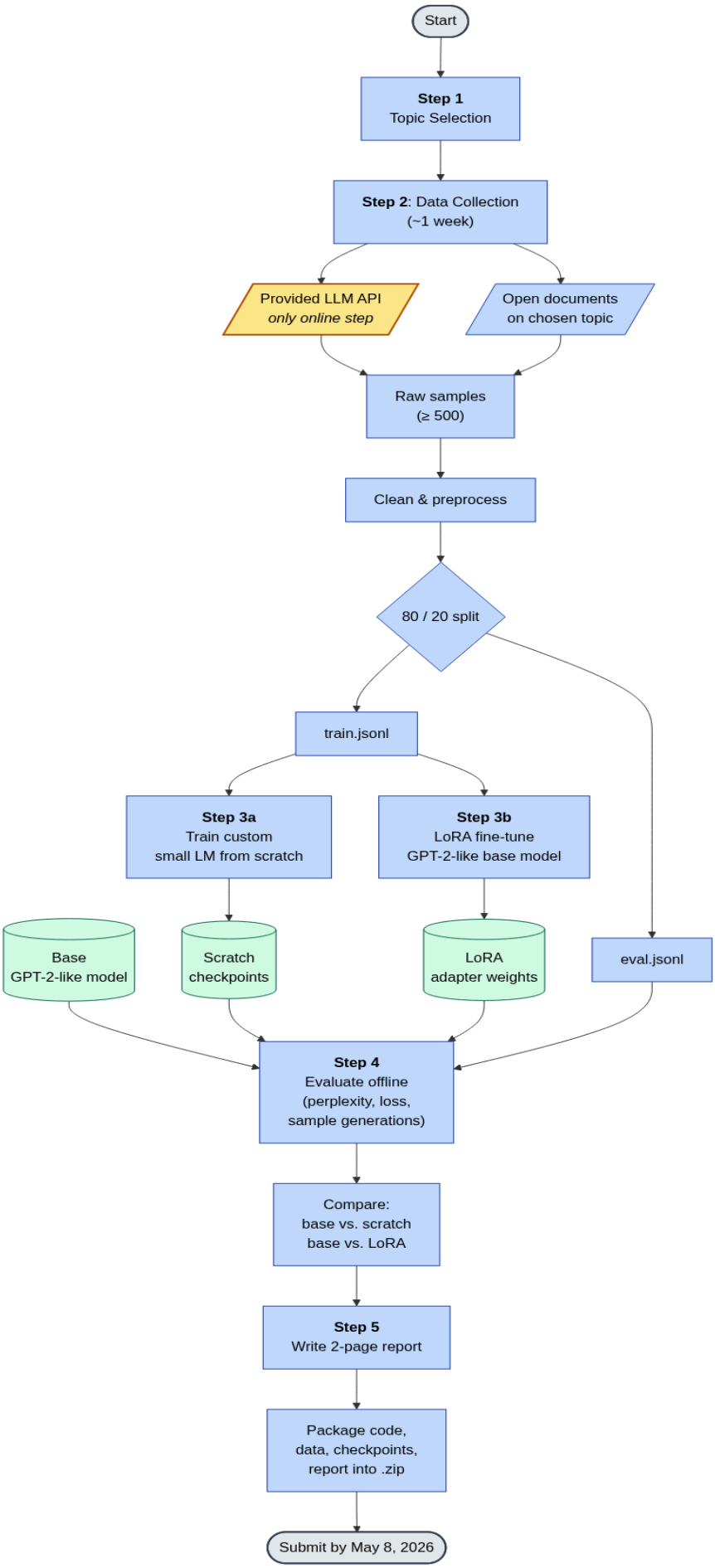
- Discussion of findings and limitations
- Potential applications and future improvements

Submission Requirements

- Written report (PDF format, 4 pages maximum, unlimited references. Template format: ACL)
- Code submission with:
  - Data collection / generation scripts (including any LLM API prompts used)
  - Processed dataset files (or a script to regenerate them offline from raw sources)
  - Implementation of the custom small language model and its training script
  - Implementation / configuration files for LoRA fine-tuning
  - Evaluation scripts
  - Generated samples from both of your models
  - A `requirements.txt` or equivalent environment file so the project can be run offline after an initial install
  - `README` with instructions to reproduce your results end-to-end offline

```
student_kit/              ←  you need to rename it to nlp2_<fullname>_<studentid>.zip
├── Makefile              ←  you talk to the kit through here
├── README.md             ←  README of the assignment
├── requirements.txt ←
│
├── reference/            ←  READ-ONLY study material
│   └── toy_rnn.py           complete LSTM LM that shows the required interface
│
├── student/              ←  YOU WORK HERE
│   ├── model.py            TODO: your custom language model
│   ├── collect_data.py     TODO: LLM-API data generation
│   ├── lora_config.py      TODO: LoRA hyperparameters
│   └── metrics.py          TODO: cross-entropy + perplexity
│
├── framework/            ←  DON'T TOUCH: training/eval plumbing
├── configs/                YAML hyperparameters (you can tune these)
├── data/raw/               drop your raw dataset here
├── checkpoints/            filled by training, 1 best checkpoint for each model only
└── outputs/                filled by evaluation
```

Please put all files in a folder and upload the `.zip` file to the submission system on Canvas.


Start
Step 1
Topic Selection
Step 2: Data Collection
(~1 week)
Provided LLM API
only online step
Open documents
on chosen topic
Raw samples
(≥ 500)
Clean & preprocess
80 / 20 split
train.jsonl
Step 3a
Train custom
small LM from scratch
Step 3b
LoRA fine-tune
GPT-2-like base model
Base
GPT-2-like model
Scratch
checkpoints
LoRA
adapter weights
eval.jsonl
Step 4
Evaluate offline
(perplexity, loss,
sample generations)
Compare:
base vs. scratch
base vs. LoRA
Step 5
Write 2-page report
Package code,
data, checkpoints,
report into .zip
Submit by May 8, 2026

Important Dates

- Release date: April 23, 2026
- Presentation and final submission deadline: May 8, 2026

Using the WARA LLM API

As an (optional) tool to generate data, we provide access to a LLM hosted on one of our servers. You can send requests there, and it will return sample text based on your input. Accessing it is done through an API call.

In what follows are the instructions for how to use this. We provide you with a small python script, where you need to substitute the access key and the model, as well as update "messages" based on what data that you need.

LLM API Information:

- BASE URL: https://warahub-be.waraml.org/api/v1/chat/completions
- ACCESS KEY: sk-ZyaaErNw4GQcNV93-TRfZA
- MODEL: *s.vu-deeptensor/llama-33-70b-instruct* OR *Qwen2-72B-Instruct-AWQ*
- NOTE: please make sure you control the number of requests per second (max 10/s) or you will get blocked.

Code sample to call API:

```
from openai import OpenAI

WARA_API_KEY = "<change-to-the-latest-key>"
def create_openai_client(api_key):
    return OpenAI(
        Base_url = API_URL,
        Api_key = api_key,
    )
def main():
    client = create_openai_client(WARA_API_KEY)  # Enter your
WARA_API_KEY
    completion = client.chat.completions.create(
        model="s.vu-deeptensor/llama-33-70b-instruct",  # Use the
model provided above
        messages=[
            {
                "role": "user",
                "content": "What is the meaning of life?"
            }
        ],
        stream=False,
        max_tokens=2048,
```

```
    )
    try:
        for chunk in completion:
            print(chunk.choices[0].delta.content, end="", flush=True)
    except:
        print(completion.choices[0].message.content)
if __name__ == "__main__":
    main()
```

For any questions related to WARAML LLMHUB, send to wara-tech@lists.cs.umu.se